

Java'da while, do-while, Jump Statements ve Comments

Koşul kontrollü loop'lar, break, continue ve kod belgeleme

Java Programlama · Dr. Öğr. Üyesi Baki Gökgöz · Gümüşhane Üniversitesi

Bu Derste Neler Öğreneceğiz?

while Loop

Tekrar sayısı bilinmediğinde: entry-controlled loop, sentinel değer.

do-while Loop

En az bir kez çalışan exit-controlled loop: menüler ve girdi doğrulama.

break

Loop'u veya switch'i anında sonlandırmak; labeled break.

continue

Mevcut turu atlayıp bir sonraki tura geçmek; labeled continue.

Comments

Single-line, multi-line ve Javadoc; iyi comment yazma alışkanlıkları.

Alıştırmalar

Trace table, çıktı tahmini ve sık yapılan hatalar.

Bölüm 1



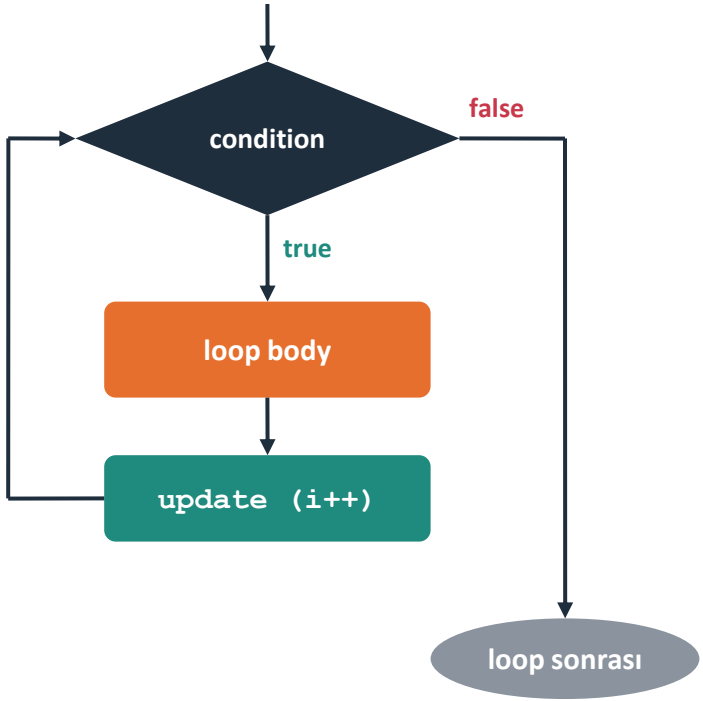
while Loop

Koşul doğru olduğu sürece tekrarla

while Loop Nasıl Çalışır?

```
Syntax
while (condition) {
    // loop body
    // update (i++)
}
```

- Condition **true** olduğu sürece body tekrar tekrar çalışır.
- **Entry-controlled loop**: condition her turdan **önce** kontrol edilir → body **hiç çalışmayabilir**.
- Initialization loop'tan **önce**, update body'nin **içinde** yapılır; update unutulursa **sonsuz loop**.
- Tekrar sayısı **önceden bilinmiyorsa** tercih edilir: kullanıcı girdisi, dosya okuma, oyun döngüsü.



while Loop: Temel Örnek

```java
int i = 1;                    // initialization
while (i <= 5) {              // condition
    System.out.println("i = " + i);
    i++;                      // update
}
System.out.println("Loop bitti, i = " + i);

int x = 100;
while (x < 10) {              // baştan false
    System.out.println("Bu satır hiç yazılmaz");
}
```

Çıktı (Output)

```
i = 1
i = 2
i = 3
i = 4
i = 5
Loop bitti, i = 6
```

- Loop bittiğinde **`i = 6`**: condition'ı false yapan ilk değer.
- İkinci loop'ta condition baştan false olduğu için body **sıfır kez** çalışır.

Örnek: Basamak İşlemleri

```java
int number = 7351;
int sum = 0, reversed = 0, digits = 0;
int n = number;
while (n > 0) {
    int digit = n % 10;       // son basamak
    sum += digit;
    reversed = reversed * 10 + digit;
    digits++;
    n /= 10;                  // son basamağı at
}
System.out.println("Basamak sayısı: " + digits);
System.out.println("Basamaklar toplamı: " + sum);
System.out.println("Ters çevrilmiş: " + reversed);
```

Çıktı (Output)

```
Basamak sayısı: 4
Basamaklar toplamı: 16
Ters çevrilmiş: 1537
```

- `n % 10` son basamağı verir, `n / 10` son basamağı atar.
- Tur sayısı sayının basamak sayısına bağlıdır; önceden bilinmediği için **while** uygundur.
- Orijinal değeri korumak için kopya (`n`) üzerinde çalışıldı.

Trace: Basamak Loop'unu Adım Adım İzlemek

Tur	n (başta)	digit = n % 10	sum	reversed	n /= 10
1	7351	1	1	1	735
2	735	5	6	15	73
3	73	3	9	153	7
4	7	7	16	1537	0
—	0	n > 0 false → loop biter			

Alıştırma: aynı tabloyu number = 4096 için doldurun. Ters çevrilmiş sayı neden 6904 olur, baştaki sıfır neden kaybolmaz?

Örnek: EBOB (Euclid) ve Collatz Dizisi

```java
int a = 48, b = 18;
while (b != 0) {  // Euclid algoritması
    int temp = b;
    b = a % b;
    a = temp;
}
System.out.println("EBOB(48, 18) = " + a);

int n = 6, steps = 0;
System.out.print(n);
while (n != 1) {  // Collatz dizisi
    n = (n % 2 == 0) ? n / 2 : 3 * n + 1;
    System.out.print(" " + n);
    steps++;
}
System.out.println("\nAdım sayısı: " + steps);
```

Çıktı (Output)

```
EBOB(48, 18) = 6
6 3 10 5 16 8 4 2 1
Adım sayısı: 8
```

- Euclid: **`EBOB(a, b) = EBOB(b, a % b)`**; b sıfır olunca a sonuçtur.
- Collatz: çiftse ikiye böl, tekse 3n+1; kaç adımda 1'e ulaşacağı önceden bilinmez.
- Her iki örnekte de **koşul**, sayaçtan değil veriden gelir.

Nested while Loop

```java
int i = 1;
while (i <= 5) {   // outer: satırlar
    int j = 1;   // her satırda sıfırla!
    while (j <= 5) {   // inner: sütunlar
        System.out.printf("%4d", i * j);
        j++;
    }
    System.out.println();
    i++;
}
```

Çıktı (Output)

```
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
   4   8  12  16  20
   5  10  15  20  25
```

- Inner loop'un sayacı (`j`) outer loop'un **içinde** sıfırlanmalıdır.
- `int j = 1;` loop dışına alınırsa inner loop yalnızca **ilk satırda** çalışır.
- Sayaçlı nested yapılarda nested **for** genellikle daha okunaklıdır.

Sonsuz Loop (Infinite Loop)

✗ Update unutuldu

```
int count = 0;
while (count < 5) {
    System.out.println("Count: " + count);
    // count++ unutuldu!
}
```

Çıktı (Output)

```
Count: 0
Count: 0
Count: 0
... (sonsuza kadar)
```

- Condition hiçbir zaman false olmaz.
- Konsolda durdurmak için **Ctrl+C**.

✓ Condition'ı değiştiren update

```
int count = 0;
while (count < 5) {
    System.out.println("Count: " + count);
    count++;
}
```

Çıktı (Output)

```
Count: 0
Count: 1
Count: 2
Count: 3
Count: 4
```

- Bilinçli sonsuz loop: **`while (true)`** + içeride **`break`** (sunucular, oyun döngüleri).

Sentinel-Controlled Loop: Kullanıcı Girdisi

```java
Scanner sc = new Scanner(System.in);
int sum = 0, count = 0;
System.out.print("Sayı girin (-1 = bitir): ");
int n = sc.nextInt();
while (n != -1) {               // sentinel kontrolü
    sum += n;
    count++;
    System.out.print("Sayı girin (-1 = bitir): ");
    n = sc.nextInt();
}
if (count > 0) {
    System.out.println("Ortalama: " + (double) sum / count);
} else {
    System.out.println("Hiç sayı girilmedi");
}
sc.close();
```

Örnek Çalışma

```
Sayı girin (-1 = bitir): 10
Sayı girin (-1 = bitir): 20
Sayı girin (-1 = bitir): 45
Sayı girin (-1 = bitir): -1
Ortalama: 25.0
```

- **Sentinel**: girişin bittiğini gösteren özel değer (-1).
- `count > 0` kontrolü 0/0 bölmesini (`NaN`) önler.
- `Scanner` için: `import java.util.Scanner;`

Bölüm 2



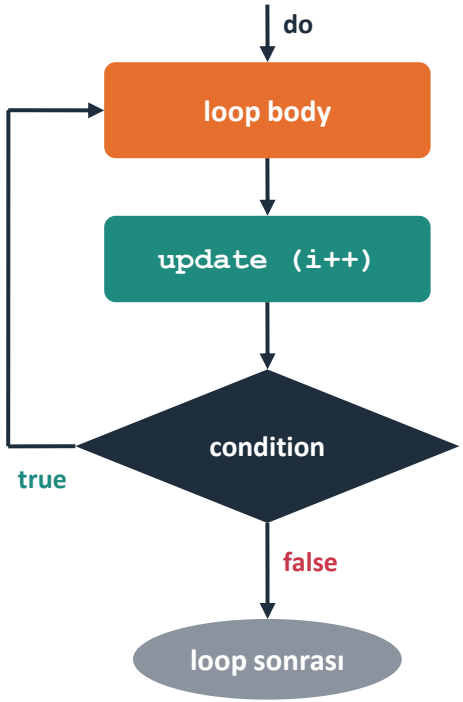
do-while Loop

Önce çalıştır, sonra kontrol et

do-while Loop Nasıl Çalışır?

```
Syntax
do {
    // loop body
    // update
} while (condition);    // ; zorunlu!
```

- **Exit-controlled loop**: condition her turun **sonunda** kontrol edilir.
- Body **en az bir kez** çalışır; condition baştan false olsa bile.
- `while (condition)` satırının sonundaki **noktalı virgül** zorunludur.
- Tipik kullanım: menüler, kullanıcıdan geçerli girdi alana kadar tekrar sorma.



while ile do-while Arasındaki Fark

while: önce kontrol

```
int i = 10;
while (i < 5) {
    System.out.println("while: " + i);
    i++;
}
System.out.println("while bitti");
```

Çıktı (Output)

```
while bitti
```

- 10 < 5 false → body **hiç** çalışmaz.

do-while: önce çalıştır

```
int i = 10;
do {
    System.out.println("do-while: " + i);
    i++;
} while (i < 5);
System.out.println("do-while bitti");
```

Çıktı (Output)

```
do-while: 10
do-while bitti
```

- Body bir kez çalışır, **sonra** kontrol edilir.

do-while: Temel Örnekler

```java
int i = 1;
do {
    System.out.print(i + " ");
    i++;
} while (i <= 10);
System.out.println();

int number = 5, factorial = 1, k = 1;
do {
    factorial *= k;
    k++;
} while (k <= number);
System.out.println(number + "! = " + factorial);
```

Çıktı (Output)

```
1 2 3 4 5 6 7 8 9 10
5! = 120
```

- Aynı işleri for ve while ile de yapabiliriz; seçim **okunabilirliğe** göre yapılır.
- Body'nin en az bir kez çalışması gerekmiyorsa while/for daha doğal okunur.

do-while ile Girdi Doğrulama (Input Validation)

```java
Scanner sc = new Scanner(System.in);
int grade;
do {
    System.out.print("Not girin (0-100): ");
    grade = sc.nextInt();
    if (grade < 0 || grade > 100) {
        System.out.println("Geçersiz! Tekrar deneyin.");
    }
} while (grade < 0 || grade > 100);
System.out.println("Kaydedilen not: " + grade);
```

Örnek Çalışma

```
Not girin (0-100): 150
Geçersiz! Tekrar deneyin.
Not girin (0-100): -5
Geçersiz! Tekrar deneyin.
Not girin (0-100): 85
Kaydedilen not: 85
```

- Kullanıcıya **en az bir kez** sormak gerektiği için do-while en doğal seçimdir.
- Geçerli değer girilene kadar soru tekrarlanır.

do-while ile Menü Uygulaması

```java
Scanner sc = new Scanner(System.in);
int choice;
do {
    System.out.println("1) Selamla  2) Yardım  0) Çıkış");
    System.out.print("Seçiminiz: ");
    choice = sc.nextInt();
    switch (choice) {
        case 1 -> System.out.println("Merhaba!");
        case 2 -> System.out.println("Bir numara girin.");
        case 0 -> System.out.println("Güle güle");
        default -> System.out.println("Geçersiz seçim");
    }
} while (choice != 0);
```

Örnek Çalışma

```
1) Selamla  2) Yardım  0) Çıkış
Seçiminiz: 1
Merhaba!
1) Selamla  2) Yardım  0) Çıkış
Seçiminiz: 5
Geçersiz seçim
1) Selamla  2) Yardım  0) Çıkış
Seçiminiz: 0
Güle güle
```

- Menü en az bir kez gösterilmelidir → do-while.
- 0 seçilince condition false olur ve program biter.

for, while ve do-while Karşılaştırması

Kriter	for	while	do-while
Condition kontrolü	Her turdan önce	Her turdan önce	Her turdan sonra
Loop türü	Entry-controlled	Entry-controlled	Exit-controlled
Minimum çalışma	0 kez	0 kez	1 kez
Ne zaman?	Tekrar sayısı biliniyorsa	Tekrar sayısı bilinmiyorsa	Bilinmiyor ve en az 1 kez gerekiyorsa
Sonsuz loop	for (;;) { }	while (true) { }	do { } while (true);
Tipik örnek	1'den 100'e toplam	Dosya sonuna kadar okuma	Menü, girdi doğrulama

Bölüm 3

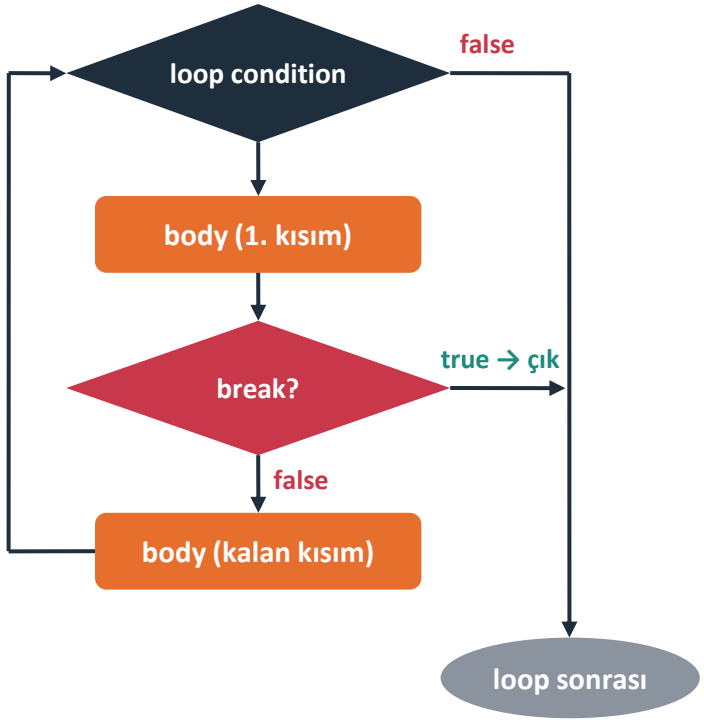


break Statement

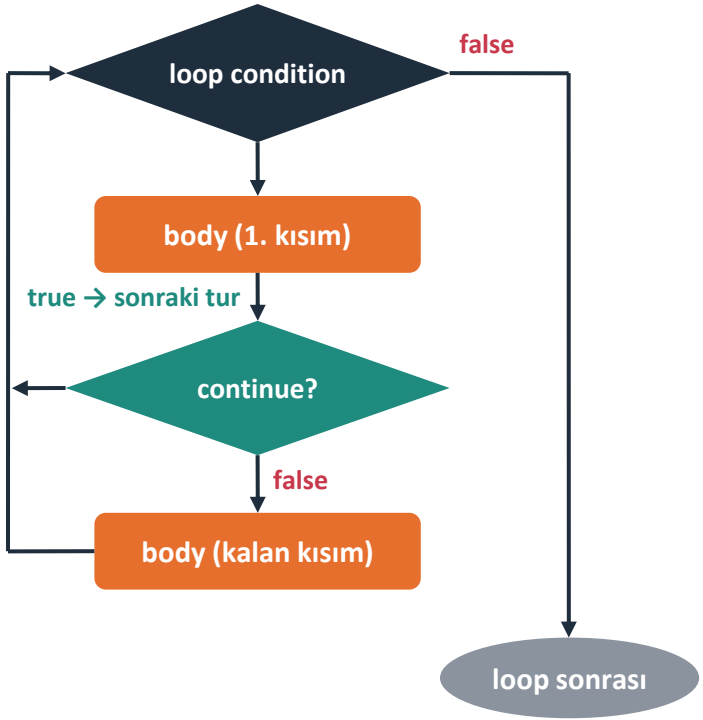
Loop'u veya switch'i erken bitirmek

break ve continue: Akış Karşılaştırması

break: loop tamamen biter



continue: sadece bu tur atlanır



break ile Arama: Hedef Bulununca Dur

```java
int[] array = {7, 2, 9, 4, 5};
int target = 9;
int foundIndex = -1;  // -1: bulunamadı
for (int i = 0; i < array.length; i++) {
    System.out.println("Kontrol: index " + i);
    if (array[i] == target) {
        foundIndex = i;
        break;  // aramayı bitir
    }
}
System.out.println("Bulunan index: " + foundIndex);
```

Çıktı (Output)

```
Kontrol: index 0
Kontrol: index 1
Kontrol: index 2
Bulunan index: 2
```

- break loop'u **anında** sonlandırır; kontrol loop'tan sonraki ilk satıra geçer.
- Index 3 ve 4 hiç kontrol edilmez → gereksiz iş yapılmaz.
- break; for, while, do-while ve switch içinde kullanılabilir; bunların dışında **compile error**.

break ile while(true) ve do-while

while (true) + break

```java
int i = 1;
while (true) {
    if (i * i > 50) {
        break;
    }
    System.out.print(i * i + " ");
    i++;
}
System.out.println("\nSonuç: " + i * i);
```

Çıktı (Output)

```
1 4 9 16 25 36 49
Sonuç: 64
```

- Çıkış koşulu loop'un **ortasında** olduğunda kullanışlıdır.

do-while + break

```java
int i = 1;
do {
    if (i == 5) {
        break;  // 5'e gelince dur
    }
    System.out.print(i + " ");
    i++;
} while (i <= 10);
```

Çıktı (Output)

```
1 2 3 4
```

- Condition hâlâ true olsa bile loop biter.

Labeled break: Nested Loop'tan Tamamen Çıkmak

```java
int[][] matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
int target = 5;
boolean found = false;
search:                                  // label
for (int r = 0; r < matrix.length; r++) {
    for (int c = 0; c < matrix[r].length; c++) {
        if (matrix[r][c] == target) {
            System.out.println("Bulundu: [" + r + "][" + c + "]");
            found = true;
            break search;                // iki loop'tan da çık
        }
    }
}
System.out.println("found = " + found);
```

Çıktı (Output)

```
Bulundu: [1][1]
found = true
```

- Etiketsiz **`break`** yalnızca **en içteki** loop'u bitirir; outer loop devam ederdi.
- Label, loop'un önüne **`isim:`** şeklinde yazılır.
- Labeled break Java'nın ilk sürümünden beri vardır.

Dikkat: Loop İçindeki switch'te break

✗ break sadece switch'ten çıkar

```java
for (int i = 1; i <= 3; i++) {
    switch (i) {
        case 2:
            System.out.println("2 bulundu");
            break;     // switch'ten çıkar
        default:
            System.out.println(i);
    }
}
```

Çıktı (Output)

```
1
2 bulundu
3
```

- Loop devam eder; 3 de yazdırılır.

✓ Labeled break loop'tan çıkar

```java
loop:
for (int i = 1; i <= 3; i++) {
    switch (i) {
        case 2:
            System.out.println("2 bulundu");
            break loop;  // loop'tan çıkar
        default:
            System.out.println(i);
    }
}
```

Çıktı (Output)

```
1
2 bulundu
```

- Alternatif: bir boolean flag veya method'dan **`return`**.

Bölüm 4



continue Statement

Bu turu atla, loop'a devam et

continue: Mevcut Turu Atlamak

```java
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        continue;                // 5'i atla
    }
    System.out.print(i + " ");
}
System.out.println();

for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue;    // çiftleri atla
    System.out.print(i + " ");
}
```

Çıktı (Output)

```
1 2 3 4 6 7 8 9 10
1 3 5 7 9
```

- continue, turun **kalan kısmını** atlar; loop **bitmez**.
- for loop'ta continue'dan sonra update (`i++`) yine çalışır.
- while/do-while'da doğrudan condition kontrolüne geçilir.

continue ile Nested Loop ve Label

continue; → inner loop'un sonraki turu

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (i == 2 && j == 2) {
            continue;
        }
        System.out.println(i + " " + j);
    }
}
```

Çıktı (Output)

```
1 1
1 2
1 3
2 1
2 3
3 1
3 2
3 3
```

- Sadece **2 2** atlanır.

continue aa; → outer loop'un sonraki turu

```
aa:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (i == 2 && j == 2) {
            continue aa;
        }
        System.out.println(i + " " + j);
    }
}
```

Çıktı (Output)

```
1 1
1 2
1 3
2 1
3 1
3 2
3 3
```

- i = 2 satırının kalanı (**2 3**) da atlanır.

Dikkat: while İçinde continue

✗ Update continue'dan sonra

```
int i = 1;
while (i <= 10) {
    if (i == 5) {
        continue;      // i++ atlanır!
    }
    System.out.println(i);
    i++;
}
```

Çıktı (Output)

```
1
2
3
4
(i hep 5 kalır → sonsuz loop)
```

✓ Update continue'dan önce

```
int i = 1;
while (i <= 10) {
    if (i == 5) {
        i++;           // önce güncelle
        continue;
    }
    System.out.println(i);
    i++;
}
```

Çıktı (Output)

```
1 2 3 4 6 7 8 9 10
(her sayı ayrı satırda)
```

Pratik Kullanım: Veri Filtreleme (Guard Clause)

```java
String[] names = {"Ali", null, "Ayşe", "", "Mehmet", null};
int printed = 0;
for (String name : names) {
    if (name == null) continue;       // null'ları atla
    if (name.isEmpty()) continue;     // boşları atla
    System.out.println(name);
    printed++;
}
System.out.println(printed + " isim yazdırıldı");
```

Çıktı (Output)

```
Ali
Ayşe
Mehmet
3 isim yazdırıldı
```

- **Guard clause**: istenmeyen durumları turun başında eleyip ana işi sade bırakmak.
- İç içe if'leri azaltır, okunabilirliği artırır.
- Çok sayıda continue ise takibi zorlaştırır; tek koşul daha iyi olabilir: **`if (name != null && !name.isEmpty())`**

Jump Statements: break, continue, return



break

- İçinde bulunduğu loop'u veya switch'i **tamamen** sonlandırır
- Labeled hali dıştaki loop'u bitirir
- Örnek: aranan eleman bulununca dur



continue

- Yalnızca **mevcut turu** atlar, loop devam eder
- Sadece loop içinde kullanılır (switch tek başına değil)
- Örnek: geçersiz veriyi atla



return

- İçinde bulunduğu **method'u** sonlandırır
- Değer döndürebilir: `return a + b;`
- main içinde programı bitirir (kaynaktaki menü örneği)

Bölüm 5



Java Comments

Kodu insanlar için açıklamak

Comment Nedir, Neden Kullanılır?

Neden?

- Compiler ve JVM tarafından **yok sayılır**
- Kodu okunabilir ve bakımı kolay yapar
- Kodu silmeden geçici olarak devre dışı bırakma

Single-line //

- `//` işaretinden satır sonuna kadar
- Kısa açıklamalar için
- En sık kullanılan tür

Multi-line /* */

- `/*` ile `*/` arasındaki her şey
- Uzun açıklamalar, kod bloklarını kapatma
- İç içe (nested) yazılamaz

Documentation /** */

- `javadoc` aracıyla HTML API dokümanı üretir
- `@param`, `@return` gibi tag'ler içerir
- public class ve method'lar için yazılır

Single-Line ve Multi-Line Comments

```java
public class CommentExample {
    public static void main(String[] args) {
        int i = 10;  // i, değeri 10 olan bir variable

        /* Aşağıdaki kod geçici olarak devre dışı:
        float j = 5.9f;
        float k = 4.4f;
        System.out.println(j + k);
        */

        System.out.println(i);  // ekrana yazdır
        // System.out.println("test");  -> çalışmaz
    }
}
```

Çıktı (Output)

```
10
```

- Kısa açıklamalar için `//`, uzun açıklamalar için `/* */` tercih edilir.
- `/* /* … */ */` gibi iç içe multi-line comment → compile error.
- IDE kısayolu: seçili satırları comment'lemek için **Ctrl + /**.

Documentation Comment (Javadoc)

```java
/**
 * Basit aritmetik işlemler yapan sınıf.
 * @author Ad Soyad
 * @version 1.0
 */
public class Calculate {
    /**
     * İki tamsayının toplamını hesaplar.
     * @param a birinci sayı
     * @param b ikinci sayı
     * @return a ile b'nin toplamı
     */
    public int sum(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println(new Calculate().sum(40, 20));
    }
}
```

Çıktı (Output)

```
60
```

- `/**` ile başlar; method/class tanımının **hemen üstüne** yazılır.
- HTML doküman üretmek için:
  - `javadoc -d docs Calculate.java`
- IDE'ler Javadoc'u kod tamamlama ve hover penceresinde gösterir.

Sık Kullanılan Javadoc Tag'leri

Tag	Syntax	Açıklama
`@author`	`@author Ad Soyad`	Class'ın yazarı
`@version`	`@version 1.0`	Sürüm bilgisi
`@since`	`@since 2026-11-01`	Hangi sürümden/tarihten beri var
`@param`	`@param a birinci sayı`	Method parametresinin açıklaması
`@return`	`@return toplam`	Dönüş değerinin açıklaması (void hariç)
`@throws`	`@throws IOException açıklama`	Fırlatılabilecek exception
`@see`	`@see String#length()`	İlgili class/method'a referans
`{@code}`	`{@code a + b}`	Metni code fontuyla, HTML yorumlamadan gösterir
`@deprecated`	`@deprecated yerine x() kullanın`	Artık kullanılmaması gereken öğe

Java 23 ve sonrasında /// ile başlayan Markdown documentation comment'leri de desteklenir (JEP 467).

İyi Comment, Kötü Comment

✗ Kodu tekrar eden comment'ler

```java
int x = 5;          // x'e 5 ata
x++;                // x'i 1 artır
// döngü
for (int i = 0; i < x; i++) { }
```

- Kodun **ne yaptığını** zaten kod söylüyor.
- Anlamsız isim (**`x`**) + comment yerine anlamlı isim kullanın.

✓ NEDEN'i açıklayan comment'ler

```java
// Sunucu geçici hata verebilir;
// 5 denemeden sonra vazgeçiyoruz.
final int MAX_RETRY = 5;
// TODO: deneme aralığını ayarlanabilir yap
int retryCount = 0;
```

- İyi kod **ne**'yi, iyi comment **neden**'i anlatır.
- **`TODO`**, **`FIXME`** etiketleri IDE'lerde listelenir.
- Kod değişince comment'i de güncelleyin.

İlginç Bir Durum: Çalışan Comment!

```java
public class Test {
    public static void main(String[] args) {
        // Aşağıdaki comment çalıştırılır!
        // \u000d System.out.println("Comment çalıştı!");
    }
}
```

Çıktı (Output)

```
Comment çalıştı!
```

- Java, Unicode escape'leri (`\uXXXX`) **lexical analizden önce** karaktere çevirir.
- `\u000d` satır sonu karakteridir: `println` comment'in dışına, yeni bir satıra düşer.
- Sadece bir merak konusu; gerçek kodda **asla** kullanmayın.

Alıştırma: Çıktı Nedir?

1

Soru 1

```
int i = 10, c = 0;
while (i > 0) {
    i -= 3;
    c++;
}
System.out.println(i + " " + c);
```

2

Soru 2

```
int n = 0;
do {
    n++;
} while (n > 5);
System.out.println(n);
```

3

Soru 3

```
for (int i = 1; i <= 10; i++) {
    if (i % 3 == 0) continue;
    if (i > 7) break;
    System.out.print(i + " ");
}
```

4

Soru 4

```
int x = 5;
/* x = x * 2; */
x += 1; // x = 100;
System.out.println(x);
```

Özet

while / do-while
while: önce kontrol (0+ tur); do-while: sonra kontrol (1+ tur). Update'i unutmayın, sonsuz loop'a dikkat.

break / continue
break loop'u veya switch'i bitirir; continue sadece turu atlar. Labeled hali dış loop'u hedefler.

Comments
// tek satır, /* */ çok satır, /** */ Javadoc. Comment kodun neden yazıldığını anlatmalı.

Çalışma önerisi
Her loop örneği için trace table doldurun; break/continue içeren kodların çıktısını çalıştırmadan önce tahmin edin.

Kaynak: tpointtech.com Java eğitim sayfaları (içerik genişletilmiş, örnekler eklenmiş ve hatalar düzeltilmiştir).